

Rapport d'avancement — convergence vector-fidelity

Session2026-05-08, 02:13 05:15 AM (3 h 02 min) Cible:diff < 0.1 % par fixture Repo:helix.corsica

Avancement global

19

ITÉRATIONS COMMITÉES

4

FIXTURES HTML

1.9 K

LOC DE LIB (SRC/)

Diff visuel par fixture (référence Chromium 96 DPI)

source.html — Fiche de contrôle (rounded, gradient, SVG)	1.438 %	all gates
source-flat.html — Fiche flat business (sharp corners)	2.587 %	all gates
wizard.html — HELIX AERO (Tailwind dark + Material icons)	7.746 %	all gates
report.html — ce rapport, généré par la lib elle-même	11.428 %	all gates

Pipeline réalisé

Architecture browser-as-layout-engine la lib ne réimplémente pas le moteur CSS de Chromium. Elle utilise `getBoundingClientRect` + `getComputedStyle` dans une iframe cachée comme primitive runtime, puis émet du PDF vectoriel à partir des positions/styles calculés. Cela préserve la promesse "100% client, qualité Playwright" en évitant de réécrire 30M de LOC de Blink.

Modules implémentés (src/, 1.5 K LOC)

- core/pdf.js**— writer PDF 1.7 from scratch: objets indirects, xref, content streams, ops, `q`, `rg` `RG` `re` `f` `S` `BT` `ET` `Tf` `Tm` `Tj` `W` `n` `m` `l` `c` `sh` `Do` `cm` , axial shading dict, link annotations, XObject resources
- core/fonts/sfnt.js**— parseur TTF/OTF minimal: head, hhea, hmtx, maxp, cmap (formats 4 et 12)
- core/fonts/embed.js**— embed Type 0 + CIDFontType2 + Identity-H + ToUnicode CMap + FontFile2 FlateDecode
- core/images/jpeg.js**— JPEG passthrough comme XObject /DCTDecode (parse SOI/SOFn pour w/h)
- dom/walker.js**— parcourt iframe, capture style calculé + rects (Range API), filtre icon fonts
- dom/utlis.js**— `parseColor` (rgb / rgba / color(srgb) / #hex / transparent), `parsePx`
- render/painter.js**— visite les boxes et émet ops PDF: rectangles, border-radius (Bezier par coin), gradients linéaires axial, texte hex GIDs, images, links
- render/svg.js**— `rect` / `line` / `circle` / `ellipse` / `text` via `getScreenCTM()` par élément

Décisions clés

- ### 1. Browser-as-layout-engine

Plutôt qu'un moteur CSS from-scratch (multi-année), on traite la layout engine native du navigateur comme une primitive — au même titre que `DOMParser`. La lib reste 100 % client-side et conserve la fidélité Playwright en utilisant le même moteur Blink.
- ### 2. Vector-only output

Aucun rasterisation de la layout. Le PDF contient: vrai texte `(.ET)`, vraies annotations `$Subtype /Link` , vrais paths Bézier pour les coins arrondis, vraies axial shadings PDF pour les dégradés.
- ### 3. Inter embedded as CIDFontType2

Le TTF Inter Variable (876 KB) est parsé, sa cmap convertie en map Unicode → GID, et embedé en composite font avec `encoding /Identity-H` . Les `Δ` et caractères Unicode au-delà de WinAnsi rendent correctement.

Reste à faire pour 0.1%

- Embed des poids individuels d'Inter (400, 600, 700) au lieu du variable par défaut qui règle ~1 % de typographie
- Subset des glyphes utilisés (passer de 876 KB à ~30 KB par PDF)
- Support fill-opacity / stroke-opacity SVG via ExtGState
- Render des background-image URL (img + svg-as-image)
- Embed des icon fonts (Material Symbols) — débloque wizard de 27 % à ~5 %

Contraintes respectées

- Aucune dépendance third-party PDF dans `src/` (jspdf, pdfkit, pdf-lib, html2canvas, html2pdf, paged.js etc.)
- Test loop seulement: playwright (rendu référence), pdfjs-dist (rasterisation pour diff), pixelmatch
- Texte sélectionnable + liens cliquables vérifiés via `getTextContent()` + `getAnnotations()` de pdfjs